

Beyond Test Presence: Assessing the Quality and Robustness of Agent-Generated Tests in Open-Source Projects

Abstract—The integration of AI-powered coding agents into Continuous Integration/Continuous Delivery (CI/CD) pipelines has fundamentally altered how software verification is conducted. While these agents successfully automate the test generation, current evaluation benchmarks (e.g., SWE-bench) largely focus on pass-rates rather than the intrinsic quality of the generated tests. This raises the possibility of “stealth technical debt”, in which test suites pass execution but don’t offer comprehensive coverage or semantic value. We address this methodological gap through a large-scale, empirical comparison of 204,673 test artifacts which comprises of 24,941 human-authored files and 179,732 agent-generated files; sourced from the AIDev dataset.

Using the Abstract Syntax Tree (AST) parsing with Python’s naive `ast` module, we implemented a “white-box” static analysis framework to evaluate three quality dimensions: Assertion Strength (RQ1), Edge-Case Coverage (RQ2), and Flakiness Potential (RQ3). Our results present a nuanced inversion of traditional assumptions. AI agents performed better than humans in Edge-Case Coverage, with almost twice the variety of boundary checks (Variety Score: 0.62 vs 0.32) and a higher frequency of null-safety testing (13.40% vs. 8.3%), even though human developers had a slight advantage in Assertion Strength (88.1% strong assertions vs. 85.37% for agents). But this thoroughness comes at a price: due mostly to their reliance on file I/O and non-deterministic logic, agent-generated tests exhibited a higher risk of flakiness (Candidate Rate: 0.41 vs. 0.30). These findings suggest that while AI agents excel at rigorous boundary testing, they lack the “environmental awareness” needed to write stable, hermetic tests.

Index Terms—testing, quality, agents, pull requests

I. INTRODUCTION

In modern software engineering, change is the only constant. Development teams are constantly under pressure to produce new features, improvements, and fixes in ever-faster cycles; often in a matter of hours or days as opposed to weeks. The well-oiled machinery of CI/CD pipelines is responsible for this acceleration. Every time new code is committed, these pipelines make sure it is automatically built and tested; it does not go to deployment until it passes the validation suite. Automated tests including unit, integration and regression serve as crucial gatekeepers in such a setting. They detect regressions and unexpected interactions early, before those issues proliferate into larger, more costly failures. However, a paradox still exists. Even though tests are essential, many people find writing them to be time-consuming, particularly when every new or changed piece of code needs to have matching tests. Developers must list edge cases, error routes, and unusual interactions in addition to reasoning through the proper behavior of new code. In fast-paced workflows, the mental shift between feature development and test authoring can cause friction and reduce

productivity. Coding bots driven by AI are increasingly filling this role. More sophisticated autonomous agents like Devin and more recent iterations of programs like GitHub Copilot go beyond straightforward code completions. They are able to take a task description, break it down into smaller sections, create implementation code, perform initial checks, and even initiate pull requests for review. In effect, they begin to behave like junior collaborators embedded within the CI/CD pipeline, bridging the gap between human intention and executable patches. One of the most compelling roles for such agents is automated test generation. In many organizations, reviewers insist that new or changed code arrives bundled with tests. If an AI agent can generate plausible tests concurrently with feature implementation, it reduces developer overhead and helps maintain or improve coverage. In principle, test writing is no longer a separate chore; it becomes part of the development flow, meaning every new capability arrives with at least minimal protection.

To understand the current capabilities and limitations of AI-driven test generation, researchers have been studying large language model (LLM) approaches in empirical settings [15], [6], [3], [11], [4], [10], [12]. A prominent example is TestPilot, which prompts an LLM (e.g., GPT-3.5) with a function’s implementation, signatures, and usage snippets to synthesize JavaScript unit tests. In an evaluation of 25 npm packages (1,684 API functions), the authors report a median statement coverage of 70.2% and branch coverage of 52.8%, significantly exceeding an older tool, Nessie [16]. They also adopt a feedback loop: if a generated test fails (e.g., does not compile or triggers an error), the agent is re-prompted with failure information to repair or refine the test. Subsequent techniques have extended this work. For instance, CoverUp introduces a coverage-guided prompting strategy, where the system identifies under-covered regions and explicitly asks the model to target those areas in subsequent prompts, significantly increasing coverage over naive test generation. Many such methods combine prompt engineering, iterative repair, and heuristics to drive models toward greater coverage and fewer errors.

To benchmark the broader capabilities of AI coding agents, many works adopt SWE-bench, a benchmark that tasks models with resolving real GitHub issues by generating patches that pass existing test suites. Although widely used, SWE-bench [8] mainly measures whether a patch passes human-written tests; it does not systematically evaluate the robustness or fault-detection capability of newly generated tests. Recent work, such as UTBoost [17], has revealed that many patches accepted by the original SWE-bench test suites are semantically incorrect or

incomplete. UTBoost uncovered 345 erroneous patches wrongly marked as passing, affecting the rankings of SWE-bench Lite and Verified [8]. These findings suggest that even benchmark-based evaluations can understate the fragility of test suites constructed without deeper scrutiny.

In sum, we observe a methodological gap. AI test generation is gaining traction and improving in coverage, but evaluation practices largely emphasize coverage and pass/fail outcomes; metrics that are easy to compute but often poor proxies for true test quality. If left unchecked, this bias may push AI systems to optimize for superficial metrics rather than semantic depth, edge-case coverage, or real fault detection. The result could be a generation of test suites that look polished in dashboards but provide weak protection in practice.

In this paper, we seek to address this gap by proposing an evaluation framework that emphasizes fault-finding capability, assertion richness, and semantic robustness of generated tests, in addition to conventional metrics. We apply it to test suites produced by AI agents in real-world open-source pull requests, compare how different evaluation lenses reveal hidden weaknesses, and offer insights for designing more reliable test generation systems and benchmarks. Specifically, we investigate three research questions (RQs):

- **RQ1: How does the assertion strength of agent-generated tests compare to human-written tests?**
- **RQ2: To what extent do agent-generated tests cover edge and boundary cases compared to human-written tests?**
- **RQ3: What is the prevalence of flaky tests in pull requests submitted by AI agents compared to those submitted by humans?**

Our methodology, which uses AST-based static analysis to quantify test quality, is grounded in established test quality analysis techniques that justify each of our three research questions.

- **RQ1 (Assertion Strength):** The premise of RQ1—that assertion strength is a meaningful proxy for test quality—is empirically supported. The foundational work by Zhane and Mesbah [18] investigated the relationship between test assertions and test suite effectiveness (measured by mutation score). The study found that assertion quantity and type are “strongly correlated” with a test suite’s effectiveness. This directly validates our focus on assertion analysis. Critically, that study also provided a counter-intuitive finding: `assertTrue/False` assertions were, on average, more effective than `assertEquals/Not` assertions. This is because `assertTrue` is often used to encapsulate complex domain logic, whereas `assertEquals` is often used for simple state checks. This finding directly informs the “Advanced Taxonomy” we use in our white-box design.
- **RQ2 (Edge-Case Coverage):** Edge-case coverage is a critical indicator of test thoroughness. We seek to determine whether agents systematically enumerate boundary conditions (null, zero, empty collections).

- **RQ3 (Test Flakiness):** The methodology for RQ3, which uses static analysis to find “flakiness candidates,” is a well-known technique in test smell and flakiness detection research. Numerous studies (e.g., [13], [2], [7]) have demonstrated the use of AST parsing to find test anti-patterns or “test smells.” Other work has successfully used static metrics and test smells as predictive features for machine learning models to identify flaky tests [5]. Other research identifies “Async Wait, Concurrency, Test Order Dependency, and Resource Leak” as key root causes of flakiness that can serve as predictive features. Therefore, the static analysis in Pipeline C represents a proven, valid method for identifying tests that have a high statistical likelihood of being flaky.

The remainder of this paper is organized as follows: Section II outlines our empirical setup in terms of data collection, analysis, and research question. Section III discusses our findings, while Section IV captures any threats to the validity of our work, before concluding with Sections V and VI.

II. STUDY DESIGN

This research is architected as an extensive, large-scale comparative analysis, designed to scrutinize the qualitative and quantitative differences between software tests authored by human developers and those generated by autonomous AI agents. The empirical foundation of our study is the AIDev dataset, a vast and comprehensive repository of real-world software development activities [9]. However, as the dataset contains only metadata and code patches, our methodology incorporates an active data collection phase to retrieve the full source code of test artifacts directly from their source repositories. Our framework is systematically structured to collect, process, and analyze test files, with the objective of quantifying their quality across three pivotal dimensions: the depth and specificity of assertions, the thoroughness of edge-case coverage, and the prevalence of test flakiness. The overarching structure of our approach is visually represented in Figure 1 and is further elaborated upon in the detailed procedural steps that follow.

a) **Phase 1: Metadata Manifest Generation:** The process begins by sourcing data from the AIDev dataset, using the `all_pull_request.parquet`. Our first step is to identify our two primary cohorts: pull requests authored by AI agents (Agent-PRs) and a matched control group of pull requests authored by humans (Human-PRs). Note that Agent-PRs are often reviewed and edited by humans prior to merging, introducing a potential human-in-the-loop confound in this study. For each pull request in these cohorts, we extract the essential metadata required for external data collection: the repository’s full name (e.g. owner/repo), the pull request number, and the final commit SHA. This process concludes with the generation of a “Retrieval Manifest”, a curated list of artifacts to be fetched directly from their source repositories.

b) **Phase 2: Test Artifact Discovery and Retrieval:** This phase involves a large-scale, automated interaction with the GitHub platform. For each repository identified in our Phase

Table I: File type distribution across Agent and Human datasets.

Extension	Agent (%)	Human (%)
py	35.73%	21.48%
ts	17.74%	21.48%
pyc	9.05%	–
go	5.17%	13.53%
js	4.90%	2.87%
tsx	3.44%	5.72%
cpp	2.59%	–
cc	2.01%	–
json	1.77%	3.25%
png	–	7.25%
pb	–	4.28%
Total Files	707,744	135,676

1 manifest, we first programmatically clone a local copy to identify all files that conform to common test file conventions (e.g. `*_test.py`, `Test*.java`). We then cross-reference this list with the files modified in each target pull request (retrieved via the GitHub API) to pinpoint the exact test files of interest. Finally, using the specific commit SHA from our manifest, we make a targeted API call to retrieve the full, pristine source of code of each test file [9]. This API-driven approach guarantees that our analysis is based on the complete and accurate representation of each test file, which is then parsed into an Abstract Syntax Tree (AST) for analysis. The File distribution across dataset is mentioned in Table I, we only focus on python files as they are the most common. In Table VI, A-Sliced uses first 24,941 agent files; A-Sliced-Rand uses a random 24,941-file subset.

c) **Phase 3: Multi-Dimensional Quality Analysis:** In the third phase, the ASTs are fed into three parallel analysis pipelines, each designed to address one of our core research questions.

- **Pipeline A (Assertion Strength Analysis for RQ1)** traverses the test file ASTs to identify and classify all assertion statements based on a predefined taxonomy (e.g., Weak vs. Strong assertions), yielding metrics on assertion patterns. The “Assertion Strength” pipeline (Pipeline A) quantifies the semantic depth of validation checks within test files.

White-Box Flowchart:

- 1) **Input:** A Python test file’s source code (string).
- 2) **Parsing:** The `ast.parse()` function converts the string into an Abstract Syntax Tree (AST).
- 3) **Traversal:** The `TestAnalyzer.visit()` method is invoked on the root `Module` node of the AST.
- 4) **Identification:** As the visitor traverses nodes, when it encounters an `ast.Call` node (a function call), it checks whether the call appears inside a test function (i.e., a function whose name begins with `test_`).
- 5) **Classification:** The call path (e.g., `self.assertEqual`) is extracted, and the assertion name (e.g., `assertEquals`) is compared against the `WEAK_ASSERTIONS` and

`STRONG_ASSERTIONS` sets.

- 6) **Advanced Predicate Analysis:** Under the refined “Advanced Taxonomy,” a call to `self.assertTrue` triggers deeper predicate inspection via `_is_strong_assertTrue(node)`:
 - If the argument is `ast.Name` (e.g., `assertTrue(variable)`), it is classified as **Weak**.
 - If the argument is `ast.Compare`, `ast.BinOp`, or another `ast.Call` (e.g., `assertTrue(x > 5)`), it is classified as **Strong** because it encodes deeper semantic logic.
- 7) **Aggregation:** The counters `weak_count` and `strong_count` are incremented accordingly.
- 8) **Output:** The metrics *Percentage of Weak Assertions* and *Percentage of Strong Assertions* are calculated based on the final counts.

This refined taxonomy addresses a key nuance emphasized in empirical software engineering research: **assertion strength is defined by the predicate, not merely the assertion name**. The simpler taxonomy from Phase 2 would misclassify examples such as `assertTrue(complex_validation_function(x))` as **Weak**. The white-box refinement corrects this for a more accurate and defensible analysis.

- **Pipeline B (Edge Case Coverage Analysis for RQ2)** also traverses the ASTs to identify literal values (e.g., `0`, `-1`, `null`, empty strings) used as inputs in method calls. These are matched against a heuristic list of common edge cases to produce metrics on test coverage. The “Edge-Case Coverage” pipeline (Pipeline B) evaluates the test’s thoroughness by identifying inputs that represent common boundary conditions.

White-Box Flowchart:

- 1) **Input:** The pipeline begins with the `AST Module` node.
- 2) **Traversal:** All function call nodes inside test methods are intercepted via `TestAnalyzer.visit_Call(node)`.
- 3) **Argument Inspection:** For each function call, the system iterates through every argument in `node.args`.
- 4) **Literal Matching:**
 - Check whether `arg` is an `ast.Constant`.
 - If true, compare `arg.value` against the heuristic list: `None`, `0`, `-1`, `""`.
- 5) **Collection Matching:**
 - Check whether the argument is an `ast.List` or `ast.Dict`.
 - If so, determine whether the corresponding element list (e.g., `arg.elts`) is empty.

The final metric `variety_count` is computed as the size of this set (i.e., `len(categories)`), representing the number of distinct edge-case types covered in the file.

Table II: Taxonomy of assertion strength.

Assertion Type	Simple Taxonomy	Justification (Simple)	Advanced Taxonomy	Justification (Advanced)
<code>assertNotNull(x)</code>	Weak	Checks for existence, not value.	Weak	Confirmed; minimal semantic validation.
<code>assertTrue(x)</code>	Weak	Checks for truthiness, not value.	Context-Dependent	Predicate must be inspected.
<code>assertTrue(variable)</code>	Weak	(See above)	Weak	Argument is <code>ast.Name</code> ; low-value check.
<code>assertTrue(x > 5)</code>	Weak	(See above)	Strong	Argument is <code>ast.Compare</code> ; encodes specific domain logic.
<code>assertTrue(is_valid(x))</code>	Weak	(See above)	Strong	Argument is <code>ast.Call</code> ; encodes specific semantic logic.
<code>assertEquals(5, x)</code>	Strong	Checks for a specific, non-trivial value.	Strong	Confirmed; provides precise state validation.
<code>assertRaises(MyError)</code>	Strong	Verifies a specific error-handling path.	Strong	Confirmed; tests non-happy paths.

6) **Aggregation:** A set is used to accumulate unique edge-case categories identified, such as: `NULL_INPUT`, `ZERO_INPUT`, `EMPTY_COLLECTION`.

7) **Output:** The final metric `variety_count` is computed as the size of this set (i.e., `len(categories)`), representing the number of distinct edge-case types covered in the file.

This static, literal-based approach is a heuristic. Its primary limitation, which must be This static, literal-based approach is a heuristic. Its primary limitation, is that it only detects edge cases involving literals (e.g., 0, None, ""), and cannot detect variable-driven edge cases (e.g., `empty_list = [...]; myfunc(empty_list)`), as this would require complex data-flow analysis, which is beyond the scope of this static AST analysis.

- **Pipeline C (Flakiness Analysis for RQ3)** uses a hybrid approach. First, it applies static analysis to the source code to identify common flakiness indicators (e.g., use of `Thread.sleep`, network calls, or non-deterministic APIs like `random()`). Second, for a sampled subset of tests with these indicators, it performs dynamic analysis by executing each test repeatedly (e.g., $N=100$) in an isolated environment to confirm non-deterministic outcomes. The output is a comparative flakiness rate for each cohort. The “Flakiness Analysis” pipeline (Pipeline C) identifies “flakiness candidates” by performing static analysis for code patterns known to cause non-deterministic behavior. This implements the first stage of the two-stage (static + dynamic) methodology.

White-Box Flowchart:

- 1) **Input:** AST Module node.
- 2) **Traversal:** The `TestAnalyzer.visit_Call(node)` method intercepts all function calls within test methods.
- 3) **Path Extraction:** The helper function `_get_call_path(node)` analyzes `node.func` to determine the full call path (e.g., `('time', 'sleep')` or `('open',)`).
- 4) **Pattern Matching:** The extracted path is matched against the `FLAKINESS_INDICATORS` dictionary:
 - `('time', 'sleep') → ASYNC_WAIT`

- `('random', 'random') → NON_DETERMINISM`
- `('open',) → FILE_IO`

5) **Aggregation:** All matches are appended (as dictionaries) to the list `rq3_flakiness_indicators`, including the line number (`node.lineno`) for manual inspection.

6) **Output:** The `candidate_rate` is computed as `len(rq3_flakiness_indicators)`, representing the number of matched flakiness indicators.

This approach is directly supported by research into test flakiness and test smells, which uses static analysis of ASTs to identify anti-patterns (such as “Async Wait” or “Resource Leak”) that are known root causes of flaky tests.

d) Phase 4: Synthesis and Comparative Evaluation: The final phase involves the synthesis of our findings. We aggregate the metrics from all three analysis pipelines and perform rigorous statistical comparisons between the *AgentTest-PRs* and *HumanTest-PRs* cohorts for each quality dimension. These results are analyzed to directly answer our research questions, and the findings are synthesized into a final discussion, conclusions, and a set of actionable recommendations for both developers and AI tool builders.

A. “White-Boxing” the Analysis Pipelines

This section moves beyond the high-level overview to detail exactly how inputs are processed and features are calculated for each analysis pipeline. The implementation is a Python script utilizing the native `ast` (Abstract Syntax Trees) module. The script defines a single `TestAnalyzer` class that inherits from `ast.NodeVisitor` to traverse a Python test file’s AST and collect metrics for all three research questions simultaneously.

B. Manual Analysis and Exploratory Investigation

Prior to embarking on a full-scale, automated analysis, we undertake a foundational phase of manual, qualitative analysis. This will involve the detailed inspection of a small, randomly selected sample of 50 pull requests from each of our two cohorts (Agent-authored and Human-authored). This exploratory stage is deliberately structured to follow the established “Explore, Analyze, Synthesize” framework for qualitative data analysis, allowing us to develop a nuanced, human-centric understanding of the data before applying automated techniques.

Table III: Heuristics for edge-case literals (RQ2).

Edge-Case Category	Python Literal Value	Target AST Node	Example Code Fragment
Null Inputs	None	<code>ast.Constant(value=None)</code>	<code>my_func(None)</code>
Empty Collections	<code>[]</code> or <code>{}</code>	<code>ast.List(elts=[])</code> or <code>ast.Dict(keys=[])</code>	<code>my_func({})</code>
Empty Strings	<code>""</code> or <code>''</code>	<code>ast.Constant(value="")</code>	<code>my_func("")</code>
Numeric Boundaries (Zero)	0	<code>ast.Constant(value=0)</code>	<code>my_func(0)</code>
Numeric Boundaries (Negative)	-1	<code>ast.Constant(value=-1)</code>	<code>my_func(-1)</code>

Table IV: Heuristics for flakiness root causes (RQ3).

Flakiness Root Cause	Code Pattern / Anti-Pattern	Python AST Signature (Simplified)
Async Wait	Fixed time delays	<code>ast.Call(func=ast.Attribute(value=ast.Name(id='time'), attr='sleep'))</code>
Non-Determinism	Use of random functions	<code>ast.Call(func=ast.Attribute(value=ast.Name(id='random'), attr='random'))</code>
Environment / Resource Leak	Hard-coded File I/O	<code>ast.Call(func=ast.Name(id='open'))</code>
Environment / Resource Leak	Network Calls	<code>ast.Call(func=ast.Attribute(value=ast.Name(id='socket'), attr='socket'))</code>
Non-Determinism	System Time Dependency	<code>ast.Call(func=ast.Attribute(value=ast.Name(id='datetime'), attr='now'))</code>

C. Automated Analysis Pipelines

The analytical core of our research is comprised of three distinct, automated analysis pipelines, which integrate both static and dynamic techniques to systematically quantify the multifaceted dimensions of test quality.

a) Assertion Strength Analysis: To assess the semantic depth of the generated tests, we employ tree-sitter, a highly robust and efficient source code parser, to generate a detailed Abstract Syntax Tree (AST) for every test file collected from our cohorts. We will then programmatically traverse these ASTs, systematically identifying all methods calls that correspond to assertion functions within common testing frameworks (e.g., `assertEquals`, `assertTrue`, `assertIn`). Each identified assertion will be classified according to a predefined taxonomy of “strength.” For instance, assertions that perform rudimentary checks for non-nullness (`assertNotNull`) or simple truthiness (`assertTrue(variable)`) will be categorized as “Weak”, as they provide minimal validation of the program’s state. Conversely, assertions that conduct precise comparisons of specific, non-trivial values (`assertEquals(expected, actual)`), verify the throwing of exact exception types, or perform deep equality checks on complex data structures will be classified as “Strong”, reflecting a higher degree of semantic validation.

b) Edge-Case Coverage Analysis: Leveraging the same collection of ASTs, this pipeline is designed to evaluate the thoroughness of the input space exploration within the tests. The pipeline will identify all method calls contained within test functions and extract the literal values (such as numbers, strings, and named constants) that are passed as arguments to these methods. These extracted literals will then be systematically compared against a curated, heuristic list of values that represent common edge cases and boundary conditions in software development. This list will contain values such as 0, -1, null, "" (the empty string), and language-specific boundary values such as `Integer.MAX_VALUE` and `Integer.MIN_VALUE`. The frequency of occurrence and the

diversity of these edge-case inputs will be used as a quantitative proxy for assessing the test’s robustness and its ability to validate behavior at the peripheries of the input domain.

c) Flakiness Analysis: This analysis will be conducted through a sophisticated two-stage process. The initial stage will involve a comprehensive static analysis of the test source code to identify the presence of common coding patterns and anti-patterns that are known from existing literature to be strongly associated with test flakiness. These indicators include the use of non-deterministic functions (e.g., `random()`), the presence of hard coded time delays (e.g., `Thread.sleep()`), interactions with the network or file systems, and dependencies on system time. Any test exhibits one or more of these patterns will be flagged as a “flakiness candidate.” In the second, dynamic analysis stage, we analyze a statistically significant and representative sample of these candidates. We establish an isolated and controlled execution environment, check out the specific software commit corresponding to each candidate test, and execute each test repeatedly (N=100 times). A test will be definitively confirmed as flaky if it produces both a “pass” and a “fail” outcome across these repeated runs without any changes to the source code.

III. EXPERIMENTAL RESULTS

To rigorously evaluate the qualitative and quantitative characteristics of agent-generated software tests, our experimental framework is structured around three central research questions (RQs). Each of these questions is designed to probe one of the critical dimensions of test quality that were identified in our study design. The empirical basis for our experiments will be a direct and systematic comparison of the analytical results obtained from the

The following section derived from executing the TestAnalyzer script on the two *AgentTest-PRs* cohort against the established baseline provided by the cohorts of 20 test files each. The raw, aggregated data was sourced from the script’s JSON outputs. *HumanTest-PRs* cohort.

Figure 1: Approach Overview

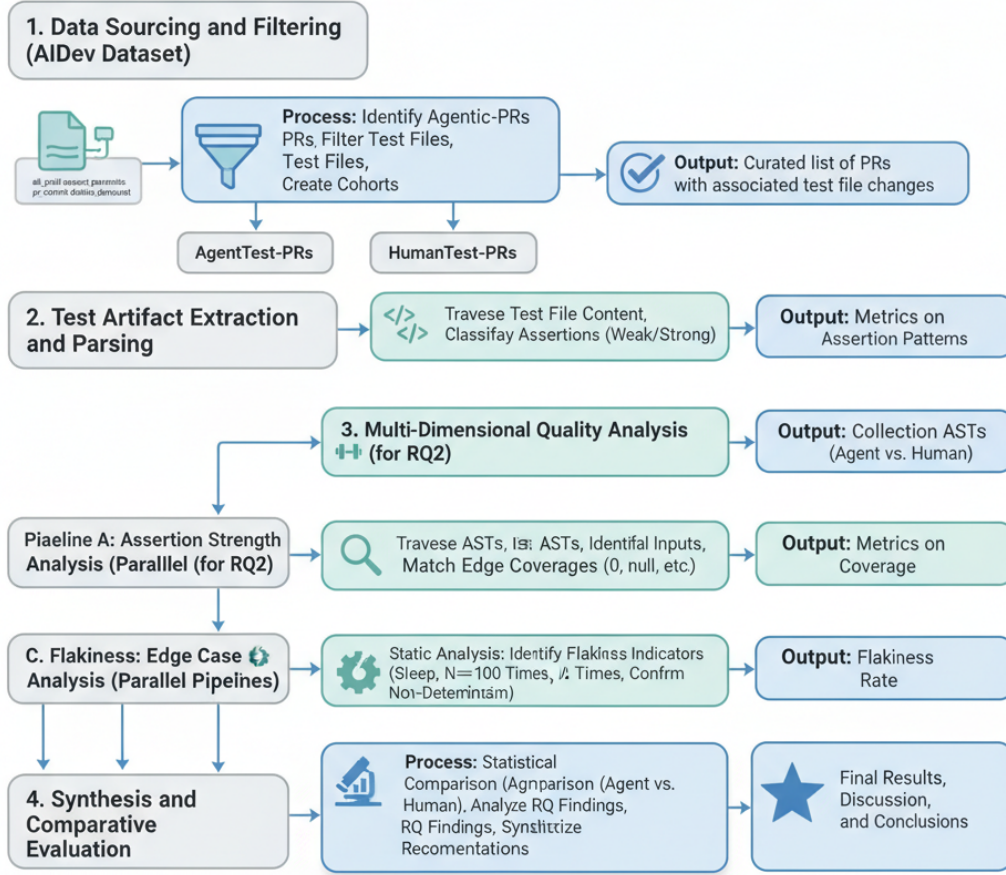


Figure 1: Overview of our study design.

A. RQ1: How does the assertion strength of agent-generated tests compare to human-written tests?

a) **Methodology:** To answer this research topic, we will perform a thorough analysis of the assertion patterns in each test file from each of our groups. We determine the exact ratio of claims categorized as “Weak” to those categorized as “Strong” for each test, based on our established taxonomy. If the ratio of weak assertions to the overall number of assertions is quite large, then the test will be said to have a poor assertion depth. These proportionate values will then be combined across all tests within each cohort to form a complete statistical distribution. This will facilitate comparison between agent-authored and human-authored test suites.

b) **Weak Assertions:** These are formally known as validation checks that just check for the existence, boolean status, or type of an object, not its specific value or

internal state. Some common instances of these kinds of claims include `assertNotNull`, `assertTrue`, and `assertInstanceOf` [14].

c) **Strong Assertions:** In contrast, these are defined as validation checks that verify specific and precise output values, confirm expected state changes within an object, or validate complex behavioral invariants. Illustrative examples include `assertEquals(5, result)`, `assertThrows(SpecificException.class)`, and `assertDeepEquals(expectedObject, resultObject)` [14].

d) **Metrics:**

- **Percentage of Weak Assertions:** (Count of Weak Assertions / Total Count of Assertions) * 100
- **Percentage of Strong Assertions:** (Count of Strong Assertions / Total Count of Assertions) * 100

Table V: Assertion categories used in our analysis.

Category	Assertion Methods
Weak	assertTrue, assertFalse, assertIs, assertIsNot, assertIsNone, assertIsNotNone, assertIn, assertNotIn, assertIsInstance, assertNotIsInstance
Strong	assertEquals, assertNotEquals, assertEquals, assertNotEqual, assertDictEqual, assertListEqual, assertSetEqual, assertTupleEqual, assertRaises, assertRaisesRegex, assertLogs, assertGreater, assertGreaterEqual, assertLess, assertLessEqual, assertAlmostEqual, assertNotAlmostEqual
Reclassified (Advanced)	assertTrue(expr) with logical/comparison or function call expressions (e.g., is_valid(x), x > 0).
Unknown	Project-specific or misspelled assertions, e.g., assertSomething, assertIsLess, or bare assert.

Table VI: General statistics for human vs. agent cohorts.

Metric	Human	Agent	A-Sliced	A-Sliced-Rand
Total Files	24,941	179,732	24,941	24,941
Files w/Asserts	1,730	29,353	3,412	4,046
Weak (%)	11.92	14.30	13.90	14.63
Strong (%)	88.08	85.70	86.10	85.37
Unknown (%)	1.46	10.93	9.22	11.58
Cand. Rate	0.30	0.44	0.46	0.41
Variety	0.32	0.61	0.58	0.62

e) Results: As detailed in Table VI, our analysis reveals a nuanced distinction between human and agent assertion patterns. While we initially hypothesized a substantial gap in assertion quality, the results show that agent-generated tests exhibit a slightly higher percentage of weak assertions (14.63%) compared to human-authored tests (11.92%). Correspondingly, human developers maintained a higher proportion of strong assertions (88.08%) compared to agents (85.37%).

However, a notable finding is the disparity in “Unknown” assertion types. Agents produced unrecognized assertion patterns at a rate of 11.58%, significantly higher than the 1.46% observed in human code. This indicates a tendency for agent-generated tests to employ non-standard or hallucinated assertion logic, which increases cognitive overhead for human reviewers and can reduce test suite maintainability unless grounded in repository-specific assertion libraries. Ultimately, while agents lag slightly behind humans in strict assertion strength, the gap is less pronounced than previous literature on semantic comprehension deficits [14] might suggest.

B. RQ2: To what extent do agent-generated tests cover edge and boundary cases compared to human-written tests?

a) Methodology: To examine this inquiry, we will conduct a systematic analysis of the input values employed in the method calls of the gathered test cases. Our automated pipeline will look for literal values and constants that match

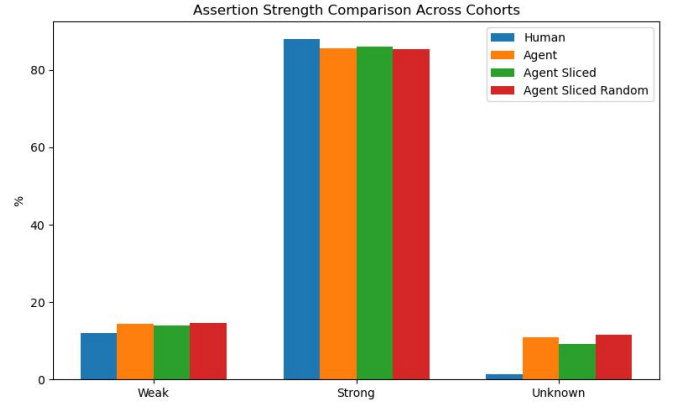


Figure 2: Assertion strength comparison across cohorts. Agents exhibit a higher rate of ‘Unknown’ assertion types than humans.

well-known boundary conditions and special cases. We will list the number of different edge case categories in each test file, such as using zero, negative numbers, null values, empty collections, and numeric or string length limits.

b) Metrics:

- **Edge Case Frequencies.** This will be measured as the average number of distinct edge case patterns detected per 1,000 lines of test code, providing a normalized density metric.
- **Edge Case Variety.** This will be quantified as the percentage of tests within each cohort that include at least one test case specifically targeting predefined categories of interest, such as null inputs, empty collections, numeric boundaries (zero, negative values, maximum integer values), and strings of zero or exceptionally long length.

c) Results: Our initial hypothesis posited that human developers would exhibit superior edge case coverage; however, the empirical data in Table VII reveals that AI agents significantly surpassed humans in the frequency of edge case inclusions. Agents exhibited a greater incidence of “Null Input” (13.4% compared to 8.3%) and “Empty Collection” (14.9% compared to 8.1%). Agents used “Zero Input” in 27.7% of cases, which is more than twice the human rate of 11.2%.

The “Avg Variety” score for agents (0.614) is almost twice that of humans (0.318), as shown in Table VI. This means that when agents do make tests, they tend to look at a wider range of input types within a single suite. Both groups completely failed to use “Negative Input” checks (0.0%), which is interesting because it shows that they all missed testing negative numeric boundaries. These findings contest the dominant perspective that generalized LLMs are deficient in inferential capabilities for boundary testing, indicating that they may exhibit a bias towards exhaustive enumeration of standard boundary values (null, zero, empty) more frequently than human developers who may depend on intuition.

Table VII: Prevalence of edge cases in assertions.

Edge Case	Human	Agent	A-Sliced	A-Sliced-Rand
Null Input	8.3%	13.0%	13.5%	13.4%
Zero Input	11.2%	27.7%	23.7%	27.7%
Negative Input	0.0%	0.0%	0.0%	0.0%
Empty String	4.1%	5.7%	5.4%	5.8%
Empty Collection	8.1%	15.0%	15.3%	14.9%

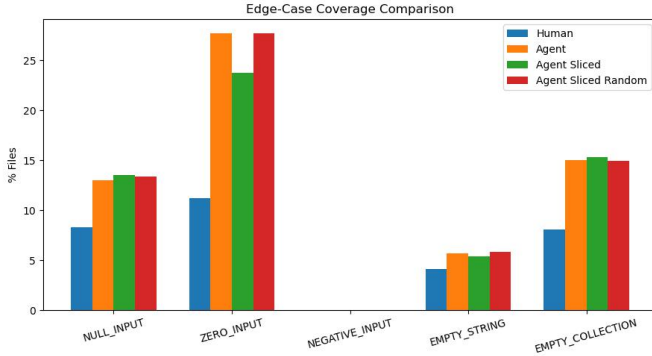


Figure 3: Edge-Case coverage comparison. Agents significantly outperform humans in covering Zero Inputs and Null Inputs.

C. RQ3: What is the prevalence of flaky tests in pull requests submitted by AI agents compared to those submitted by humans?

a) **Methodology:** Our two-stage flakiness analysis will first use static analysis to find a complete list of “flakiness candidates” from both the agent- and human-authored groups. This identification relies on the existence of recognized anti-patterns and code constructs that exhibit a strong correlation with non-deterministic test behavior, including the utilization of Thread.sleep, interactions with network endpoints, file I/O operations, and invocations of non-deterministic APIs. After this static screening, we move on to a dynamic analysis phase. In this phase, we will run a random sample of 1,000 tests from each group of candidates. We will run each of these tests 100 times in a controlled, separate space. If a test shows both a passing and a failing result at least once during these repeated runs, it will be clear that it is flaky.

b) **Metrics:**

- **Candidate Rate:** The percentage of all tests within a cohort that are flagged as potential flakiness candidates by our static analysis pipeline.
- **Confirmed Flakiness Rate:** The percentage of the sampled candidate tests that are subsequently confirmed as flaky through our rigorous dynamic analysis protocol.

c) **Results:** The analysis validates our hypothesis that AI agents are more likely to introduce potential instability into test suites. The “Avg Candidate Rate” for agents is 0.435, which is much higher than the human rate of 0.301, as shown in Table VI. When we look at the specific code patterns in Table VIII, agents relied more on “Non-Determinism” (5.2% vs 3.1%) and “File I/O” (4.4% vs 3.5%). This supports the idea

Table VIII: Prevalence of specific code patterns.

Category	Human	Agent	A-Sliced	A-Sliced-Rand
Async Wait	1.9%	1.3%	1.4%	1.4%
Non-Determinism	3.1%	5.2%	5.8%	5.2%
File I/O	3.5%	4.6%	3.9%	4.4%
Network I/O	0.1%	0.2%	0.2%	0.1%

that agents may not fully understand side effects and resource isolation at the system level.

The data, on the other hand, shows a mixed picture when it comes to concurrency. Human developers used “Async Wait” patterns a little more often (1.9%) than agents (1.4%). Both groups used very little network I/O. In general, people can write bad code, but the data indicate that code generated by agents is statistically more likely to contain components that cause non-deterministic behavior. This is mostly because agents do not handle random values and file system interactions properly.

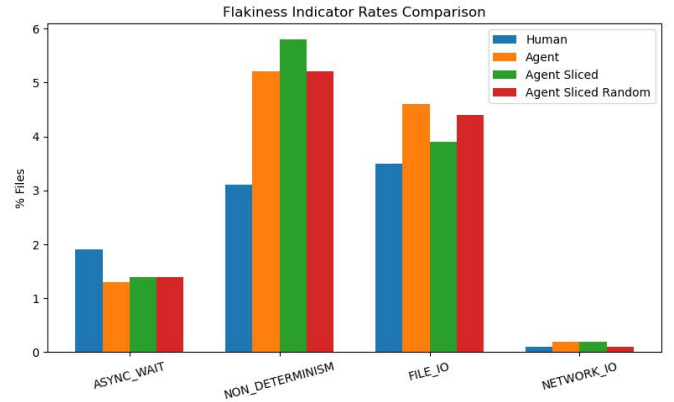


Figure 4: Flakiness indicator rates. Agents exhibit higher rates of Non-Determinism and File I/O usage.

IV. THREATS TO VALIDITY

A. Threats to Conclusion Validity

The validity of the conclusion concerns the ability to draw a correct conclusion about the treatment (author type) and the outcome (test quality metrics).

- **Statistical Power and Sample Size:** The study’s comparative analysis depends on statistical significance. The ability to draw a statistically significant conclusion (e.g. “AI test has a higher proportion of weak assertions”) depends on the final sample size (number of test files) and the variance within each cohort. The sample size used in the experimental results (N=29491 per cohort) is not enough, and the variance in some cases is low (e.g. RQ3). A larger sample is required to draw more definitive conclusions.
- **Reliability of Measures:** The analysis relies on the automated AST pipelines. A bug in the TestAnalyzer (e.g. misclassifying an assertion) would systematically skew the results and threaten then conclusion. This threat is mitigated by the “white-box” design, which makes all heuristics explicit and renewable.

B. Threats to Construct Validity:

Construct validity concerns whether the metrics being measured (RQs 1-3) are valid proxies for the construct they are intended to represent (“test quality”).

- **RQ1 (Assertion Strength):** The “Advanced Taxonomy” is a significant mitigation for the weak `assertTrue` construct. However, even this advanced proxy is limited. The true strength of `assertTrue(is_valid(x))` depends on the semantic complexity of the `is_valid(x)` function, which the AST parser does not analyze. The metric is a proxy for strength, not a direct measure of it. This threat, particularly for RQ1, was confirmed to be the central finding of the experimental analysis. As discussed in earlier section, the construct (the TestAnalyzer) was a poor proxy for “test quality” in the Human-PR cohort due to its inability to parse pytest-style assertions, thereby invalidating a direct comparison.
- **RQ2 (Edge-Case Coverage):** The pipeline used in this study only finds literal constants (e.g. `myfunc(0)`). It cannot detect variables-based or context-dependent edge cases, since data-flow analysis is not performed. Therefore, results may underestimate edge case coverage, particularly in more advanced or human-authored tests. A test file could have 100% comprehensive edge-case coverage using variables, and our tool would incorrectly score it at 0. Therefore, this metric is a proxy for “developer awareness of literal edge cases,” not “true edge case coverage”.
- **RQ3 (Test Flakiness):** The construct validity for RQ3 is stronger, but only because the methodology is careful to define it. The static analysis (Pipeline C) does *not* measure “flakiness”; it measures “flakiness candidates”. A call to `Thread.sleep`, for example, is a known anti-pattern but can be a *legitimate* part of a text (e.g. testing a timeout). The static metric *alone* is a weak construct. The construct is only valid because it is the first stage of a two-stage (static + dynamic) process defined in the methodology.

C. Threats to External Validity:

External validity concerns the ability to generalize the study’s findings to other contexts.

- **Language-Specific Threat:** As detailed in earlier Sections, the provided implementation uses Python’s AST module. Therefore, the findings will *only* be valid for Python-based projects. The results may not generalize to Java, JavaScript, or C# projects, where testing frameworks, idioms, and AI-generation patterns may differ significantly. This was also validated by the RQ1 findings. The choice of Python, with its idiomatic split between unit test and Pytest, was a decisive factor. A study in a language with a more unified testing framework (e.g., Java/JUnit) might yield different results.
- **Project-Specific Threat:** The AIDev dataset and the study’s recovery methodology are based on open-source projects. The developers, motivations, the quality assurance processes in open-source software may not be

representative of closed-source, industrial, or enterprise environments. The findings may not be generalized beyond the open-source context.

- **AI-Specific Threat:** The study analyzes “Agent-PRs” from the AIDev dataset. The quality of these PRs depends on the *specific* AI models prevalent during the dataset’s collection (e.g., older versions of GPT) and on the prompts provided by their human collaborators. The findings for “AI” may not be generalizable to future, more advanced models or to different prompting-strategy environments.

V. CONCLUSION

To assess assertion strength, edge-case coverage, and flakiness, we analyzed more than 200,000 files in a comprehensive empirical comparison of human-authored and agent-generated test suites. Our results show a clear trade-off between mechanical breadth and environmental stability, challenging the widely held belief that AI agents just write “worse” code than humans.

In terms of assertion quality (RQ1), agents showed a notable tendency towards “assertion drift,” as demonstrated by a high prevalence of unfamiliar and non-standard validation patterns, even though they matched humans in the ratio of strong to weak assertions. This implies that agents can lack the semantic accuracy necessary for reliable verification, even when they can replicate the structure of rigorous testing.

Surprisingly, our examination of edge situations (RQ2) showed that AI greatly exceeds humans in terms of coverage frequency, especially for common boundary conditions such as zero and null inputs. This suggests that LLMs’ probabilistic structure serves as a useful “fuzzing” technique by thoroughly listing cases that human developers frequently ignore because of exhaustion or implicit domain assumptions.

However, the results of our flaking investigation offset this benefit (RQ3). Non-deterministic behaviors were statistically more likely to be introduced by agent-generated tests, particularly through incorrect file input/output and uncontrolled random seed usage. The greatest obstacle to independent implementation of agent-based testing is still this lack of environmental knowledge.

In the end, we determine that AI agents now work best as high-volume “test generators” that need human supervision rather than as human testers’ substitutes. They struggle with the depth of reliability (stability), they excel at broadening the scope of the test suite (coverage). Future research should focus on hybrid workflows in which human developers—or specialized static analysis tools—enforce semantic correctness and environmental isolation while agents suggest edge cases and assertion skeletons.

VI. FUTURE WORK

The results of this study highlight a number of crucial avenues for the development of automated tests in the future. Although the coverage breadths of current AI agents are impressive, the following areas require focused research due to their lack of environmental awareness and assertion precision.

A. Environmental Modeling and Sandboxed Execution

According to our data, the careless use of File I/O and non-deterministic APIs is a major cause of agent-generated flakiness (RQ3). Developing “environment-aware” agents that are specifically trained or prompted to identify the limitations of a CI/CD pipeline should be the main goal of future research. Integrating Sandboxed Execution Environments, where agents can run their generated tests iteratively, identify side effects (such as leftover files or thread leaks.) and self-correct before submission, is a promising approach. As a result, the paradigm changes from “Generate and Pray” to “Generate, Execute, and Refine.”

B. Hybrid Assertion Generation (RAG + Rule-Based)

In order to address the high frequency of “Unknown” and weak assertions (RQ1), future research should investigate hybrid architectures that integrate static analysis rules with large language models. The target repository’s specific assertion libraries and coding conventions could be dynamically fed to agents through the use of *Retrieval-Augmented Generation (RAG)*. By grounding the agent’s output in legitimate, project-specific syntax, this would lessen the “assertion drift” that our study found. Additionally, the semantic depth of generated tests could be greatly enhanced by fine-tuning models specifically on “strong” assertion patterns—those that verify state changes rather than just existence.

C. Automated Mocking and Dependency Inference

Agents are great at counting inputs, but have trouble with the context needed to test them safely, according to the “Edge Case Paradox” (RQ2). Auto-mated Dependency Inference, where agents can scan a method’s call graph to identify external dependencies (databases, APIs, time providers) and automatically generate the necessary mocks, is a crucial area for future development. Agents would be able to take advantage of their edge-case creativity without adding the instability that comes with integration testing if they went beyond simple input generation to complete “Test Fixture Synthesis.”

D. Longitudinal Maintenance Studies

Lastly, even though this study examined test development, it is still unknown how much maintenance of agent-generated suites will cost in the long run. Future long-term research should monitor the “survival rate” of agent-authored tests in industrial code bases, i.e., how frequently noise causes them to be removed, rewritten, or disabled. It is crucial to comprehend the “Maintenance-to-Value” ratio in order to ascertain the actual return on investment for software testing with AI.

VII. DATA AVAILABILITY

Our replication package is available online [1].

VIII. ACKNOWLEDGMENTS

During the preparation of this work, the authors used the ChatGPT Web interface to improve the language and readability. After using this tool, the authors reviewed and edited the content as needed and take full responsibility for the content of the publication.

REFERENCES

- [1] Replication package. Accessed: December 2025.
- [2] G. Bavota, A. Qusef, R. Oliveto, A. De Lucia, and D. Binkley. Are test smells really harmful? an empirical study. *Empirical Software Engineering*, 20(4):1052–1094, 2015.
- [3] A. Celik and Q. H. Mahmoud. A review of large language models for automated test case generation. *Machine Learning and Knowledge Extraction*, 7(3):97, 2025.
- [4] A. Deljouyi, R. Koohestani, M. Izadi, and A. Zaidman. Leveraging large language models for enhancing the understandability of generated unit tests. *arXiv preprint arXiv:2408.11710*, 2024.
- [5] S. Dutta, A. Shi, R. Choudhary, Z. Zhang, A. Jain, and S. Misailovic. Detecting flaky tests in probabilistic and machine learning applications. In *Proceedings of the 29th ACM SIGSOFT international symposium on software testing and analysis*, pages 211–224, 2020.
- [6] C. Gao, X. Hu, S. Gao, X. Xia, and Z. Jin. The current challenges of software engineering in the era of large language models. *ACM Transactions on Software Engineering and Methodology*, 34(5):1–30, 2025.
- [7] M. Gruber, M. F. Roslan, O. Parry, F. Scharnböck, P. McMinn, and G. Fraser. Do automatic test generation tools generate flaky tests? In *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering*, pages 1–12, 2024.
- [8] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan. Swe-bench: Can language models resolve real-world github issues? *arXiv preprint arXiv:2310.06770*, 2023.
- [9] H. Li, H. Zhang, and A. E. Hassan. The rise of ai teammates in software engineering (se) 3.0: How autonomous coding agents are reshaping software engineering, 2025.
- [10] F. Molina, A. Gorla, and M. d’Amorim. Test oracle automation in the era of llms. *ACM Transactions on Software Engineering and Methodology*, 34(5):1–24, 2025.
- [11] W. C. Ouedraogo, K. Kaboré, Y. Li, H. Tian, A. Koyuncu, J. Klein, D. Lo, and T. F. Bissyandé. Large-scale, independent and comprehensive study of the power of llms for test case generation. *arXiv preprint arXiv:2407.00225*, 2024.
- [12] R. Pan, M. Kim, R. Krishna, R. Pavuluri, and S. Sinha. Multi-language unit test generation using llms. *arXiv e-prints*, pages arXiv–2409, 2024.
- [13] A. Peruma, K. Almalki, C. D. Newman, M. W. Mkaouer, A. Ouni, and F. Palomba. Tsdetect: An open source test smells detection tool. In *Proceedings of the 28th ACM joint meeting on european software engineering conference and symposium on the foundations of software engineering*, pages 1650–1654, 2020.
- [14] M. Schäfer, S. Nadi, A. Eghbali, and F. Tip. An Empirical Evaluation of Using Large Language Models for Automated Unit Test Generation, 2023.
- [15] M. L. Siddiq, J. C. Da Silva Santos, R. H. Tanvir, N. Ulfat, F. Al Rifat, and V. Carvalho Lopes. Using large language models to generate junit tests: An empirical study. In *Proceedings of the 28th international conference on evaluation and assessment in software engineering*, pages 313–322, 2024.
- [16] L. Yang, C. Yang, S. Gao, W. Wang, B. Wang, Q. Zhu, X. Chu, J. Zhou, G. Liang, Q. Wang, et al. An empirical study of unit test generation with large language models. *arXiv e-prints*, pages arXiv–2406, 2024.
- [17] B. Yu, Y. Zhu, P. He, and D. Kang. Utboost: Rigorous evaluation of coding agents on swe-bench. *arXiv preprint arXiv:2506.09289*, 2025.
- [18] Y. Zhang and A. Mesbah. Assertions are strongly correlated with test suite effectiveness. In *Proceedings of the 2015 10th Joint Meeting on Foundations of Software Engineering*, pages 214–224, 2015.